

VirtuEx 성능 개선 감사 보고서 (9차)

VirtuEx Performance Improvement Audit Report (9th)

- 감사일: 2026-03-20
- 감사 범위: 프론트엔드 렌더링/Memoization, 번들/코드 분할, API/네트워크 효율, 캐싱/메모리 관리, DB/서버 성능, WebSocket/실시간 처리, 리스트/가상화, 기타
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음
- **9차 강화 기준:** 컴포넌트 200줄 초과 플래그(8차: 300줄), 20건 이상 리스트 가상화 필수, 모든 비동기 작업 로딩 상태 검사, useEffect 의존성 정확성 감사, prop drilling 검사

이전 감사 대비 수정 현황 (8차 → 9차)

이전 이슈	상 태	비고
[PF-H-01] 뉴스 AI 분석 200건 과잉 조회	미 수 정	dashboard/page.tsx:178, news/page.tsx:157 — 여전히 limit: 200으로 전체 조회 후 50건 슬라이싱
[PF-M-01] AssetList currentSymbols map+join 매 렌더	미 수 정	AssetList.tsx:98 — 여전히 assets.map(a => a.symbol).join(', ') 매 렌더 실행
[PF-M-02] 주문 분석 탭 trades.filter() 반복	부 분 수 정	AnalysisTab으로 별도 컴포넌트 분리. 그러나 breakdown 내 trades.filter() O(N*M) 패턴 유지 (189줄)
[PF-L-01] PortfolioHistoryChart totalValue 재계산	미 수 정	PortfolioHistoryChart.tsx:128 — useMemo 의존성에 totalValue 포함 그대로
[API-H-01] 뉴스 필터 200건 일괄 조 회	미 수 정	FILTERED_FETCH_LIMIT = 200, 서버 측 search/date 파라미터 미전달
[API-M-01] 체결 내역 프론트엔드 서 버 페이지네이션 미활용	미 수 정	useTradeHistory() 여전히 limit/offset 미전달
[API-M-02] 포트폴리오 valuation/summary 동시 폴링	부 분 수 정	portfolio/page.tsx에서 usePortfolioValuation()만 사용, 그러나 OrderForm.tsx 등 다른 컴포넌트에서 usePortfolio() 별 도 폴링 가능
[BD-M-01] 대시보드 516줄 단일 컴포 넌트	미 수 정	TODO 주석 유지, 516줄 그대로

[BD-L-01] 주문 페이지 인라인 분석 로직	수정됨	AnalysisTab 별도 컴포넌트로 추출 완료. 단, dynamic import 미 적용
[CM-M-01] CandlestickChart chartType 변경 시 8개 시리즈 전체 재 생성	미수정	CandlestickChart.tsx:319-404 동일 패턴 유지
[CM-M-02] 리더보드 sortedLeaderboard 필터 변경 시 전체 재정렬	미수정	leaderboard/page.tsx:212-230 동일
[CM-L-01] 리더보드 prevRankMap 기간/정렬 변경 시 미초기화	미수정	leaderboard/page.tsx:235 동일
[DB-M-01] 관리자 체결 감사 사용자명 enrichment N+1	미수정	order-proxy.controller.ts:162-179 동일
[DB-L-01] 거래 내역 limit=200 하드 코딩	미수정	useTransactions(limit = 200) 동일
[WS-M-01] useWebSocket symbols 변경 시 전체 목록 재전송	미수정	useWebSocket.ts:196-198 subscribe 시 전체 목록 전송
[WS-L-01] 종목 상세 단일 심볼 배열 래핑	미수정	실질 영향 미미로 보류

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 렌더링/Memoization	0	1	3	1	5
B. API/네트워크	0	1	2	1	4
C. 번들/코드 분할	0	1	3	1	5
D. 캐싱/메모리	0	0	2	1	3
E. DB/서버	0	0	1	1	2
F. WebSocket/실시간	0	0	1	0	1
G. 리스트/가상화	0	0	2	0	2
H. 기타	0	0	2	1	3
합계	0	3	16	6	25

A. 렌더링/Memoization — 5건

[PF-H-01] 대시보드/뉴스 AI 분석 — 뉴스 200건 전체 fetch 후 50건 슬라이싱 (미수정, 3차 연속)

- **현상:** dashboard/page.tsx:178 과 news/page.tsx:157 에서 `api.get('/api/news', { params: { limit: 200 } })` 로 200건 전체를 가져온 후 24시간 내 50건만 사용. 3차 연속 미수정 상태로 심각도 High 유지
- **위치:** frontend/src/app/(main)/dashboard/page.tsx:178 , frontend/src/app/(main)/news/page.tsx:157
- **영향:** 불필요한 네트워크 전송량(최대 4배 과잉), 서버 DB 쿼리 부하, 모바일 환경에서 파싱 지연
- **권장:** 서버 API에 `publishedAfter` 파라미터를 추가하여 24시간 내 뉴스만 조회하거나, `limit`을 50으로 줄이고 서버 측 날짜 필터 적용. 두 페이지 모두 동일 패턴이므로 공유 유틸로 통합
- **심각도:** High

[PF-M-01] AssetList currentSymbols — assets.map().join() 매 렌더 실행 (미수정, 3차 연속)

- **현상:** AssetList.tsx:98 에서 `useMemo(() => assets.map(a => a.symbol).join(','), [assets])` . `assets` 참조는 WebSocket 틱마다 새로 생성되므로 매번 100개+ 심볼을 map+join 실행. `assetsVersion` `useMemo`에서 이전 문자열과 비교하여 정렬 재실행을 방지하지만, join 자체의 O(N) 연산은 매번 발생
- **위치:** frontend/src/components/market/AssetList.tsx:98-108
- **영향:** 500ms마다 100+ 심볼의 join 및 문자열 비교 수행. 9차 강화 기준에서 불필요한 O(N) 연산으로 Medium 유지
- **권장:** `assets.length` 와 첫/마지막 심볼만 O(1) 비교하거나, 상위 컴포넌트에서 심볼 Set의 안정적 ref 전달
- **심각도:** Medium

[PF-M-02] 주문 분석 탭 breakdown — trades.filter() O(N*M) 반복 (부분 수정)

- **현상:** orders/page.tsx:189 의 AnalysisTab 내 `stats` `useMemo`에서 `symbolMap` 으로 `buyTotal/sellTotal`을 이미 집계했음에도, `breakdown` 계산 시 `trades.filter(tr => tr.symbol === symbol && tr.buyerId === userId)` 호출로 O(N*M) 복잡도 발생. 컴포넌트 분리는 완료되었으나 핵심 성능 이슈 미해결
- **위치:** frontend/src/app/(main)/orders/page.tsx:189
- **영향:** 체결 내역 1000건 × 심볼 50종 시 50,000번 필터 반복
- **권장:** `symbolMap` 순회 시 `buyCount/sellCount`를 동시 집계하여 filter 호출 제거
- **심각도:** Medium

[PF-M-03] PortfolioHistoryChart — totalValue 변경 시 전체 시계열 재구성 (미수정, 3차 연속)

- **현상:** PortfolioHistoryChart.tsx:127-130 에서 `buildHistoryFromTransactions` 의 `useMemo` 의존성에 `totalValue` 가 포함. 포트폴리오 가격이 10초 폴링으로 변할 때마다 전체 거래 내역을 재정렬하고 시계열 데이터를 재구성
- **위치:** frontend/src/components/portfolio/PortfolioHistoryChart.tsx:127-130
- **영향:** 거래 내역 200건+ 시 10초마다 불필요한 배열 복사/정렬 반복
- **권장:** 거래 내역 기반 시계열은 `transactions / period` 변경 시에만 계산하고, `totalValue` 변경 시에는 마지막 포인트만 교체
- **심각도:** Medium

[PF-L-01] 뉴스 검색 — searchQuery 디바운스 미적용

- **현상:** news/page.tsx:269 에서 `onChange={(e) => setSearchQuery(e.target.value)}` 로 키 입력마다 즉시 상태 업데이트. `filteredItems` `useMemo`가 `searchQuery` 의존성으로 매 키 입력마다 200건 필터링 재실행. 주문 페이지의 `symbolSearch` 는 300ms 디바운스를 적용했지만 뉴스 페이지는 미적용
- **위치:** frontend/src/app/(main)/news/page.tsx:269
- **영향:** 사용자가 빠르게 타이핑 시 매 글자마다 200건 배열 필터링

- 권장: 주문 페이지와 동일한 300ms 디바운스 패턴 적용
 - 심각도: Low
-

B. API/네트워크 효율 — 4건

[API-H-01] 뉴스 필터 200건 일괄 조회 후 클라이언트 페이지네이션 (미수정, 3차 연속)

- 현상: `news/page.tsx:80-81` 에서 검색어나 날짜 필터가 적용되면 `FILTERED_FETCH_LIMIT = 200` 으로 한번에 200건을 조회한 후 클라이언트에서 필터링+페이지네이션 수행. 서버 API에 `search` , `publishedAfter` 파라미터 미전달
- 위치: `frontend/src/app/(main)/news/page.tsx:78-81`
- 영향: 불필요한 대량 데이터 전송, 서버 쿼리 부하, 모바일 데이터 낭비
- 권장: 서버 API에 `search` , `dateFilter` 파라미터 추가. 뉴스 프록시 컨트롤러(`news-proxy.controller.ts`)가 이미 `req.query` 를 그대로 전달하므로 백엔드 `news.service`에 검색/날짜 필터 로직만 추가하면 해결
- 심각도: High

[API-M-01] 체결 내역 — 프론트엔드에서 서버 페이지네이션 미활용 (미수정, 3차 연속)

- 현상: `useTradeHistory()` 혹(`useOrders.ts:194-219`)이 `/api/orders/trades/history` 에 `limit / offset` 파라미터를 전달하지 않고 전체 체결 내역을 가져옴. 백엔드 `order-proxy.controller.ts:227-230` 에서 이미 `limit / offset` 지원 확인. 마이페이지 (`mypage/page.tsx:139`)에서도 동일 혹 사용하여 전체 데이터 로드
- 위치: `frontend/src/hooks/useOrders.ts:197-198`
- 영향: 체결 내역 누적 시 응답 크기 선형 증가. 마이페이지에서는 통계 집계용이므로 전체 데이터가 필요하나, 주문 페이지 체결 탭에서는 페이지네이션 적용 필요
- 권장: `useTradeHistory(limit?, offset?)` 파라미터 추가. 주문 체결 탭에서 페이지 단위 요청, 분석 탭에서는 서버 측 집계 API 제공
- 심각도: Medium

[API-M-02] `getDateCutoff()` — `useMemo` 외부에서 호출되어 매 렌더마다 `Date` 객체 생성

- 현상: `news/page.tsx:112-118` 의 `getDateCutoff()` 함수가 `filteredItems` `useMemo` 내부 (:126)에서 호출됨. 함수 자체는 컴포넌트 본문에 정의되어 매 렌더마다 새로 생성되지만, `useMemo` 의존성에 `dateFilter` 가 포함되어 있으므로 실질적으로 `dateFilter` 변경 시에만 실행됨. 그러나 `getDateCutoff` 가 `useMemo` 의존성에 미포함되어 linter 경고 가능성 및 의도와 다른 동작 위험
- 위치: `frontend/src/app/(main)/news/page.tsx:112-118, 126`
- 영향: 현재는 동작에 문제 없으나, React 엄격 모드나 향후 변경 시 stale 클로저 위험
- 권장: `getDateCutoff` 를 `useMemo` 또는 `useCallback` 으로 감싸거나, `dateFilter` 를 직접 `cutoff` 계산에 인라인
- 심각도: Medium

[API-L-01] 리더보드 `period/sortBy` — 서버 미지원으로 클라이언트 전체 재정렬

- 현상: `useLeaderboard.ts:49` 에서 `api.get('/api/portfolio/leaderboard', { params: { limit: 100 } })` 로 항상 100명만 조회. `period` 와 `sortBy` 가 `queryKey`에는 포함되지만 서버 파라미터로 미전달 — 모든 기간/정렬 조건에서 동일 데이터를 조회하고 클라이언트에서 재정렬/필터링
 - 위치: `frontend/src/hooks/useLeaderboard.ts:49`
 - 영향: 기간 변경 시에도 동일 데이터를 반복 요청, 서버 측 기간별 수익률 계산 미활용
 - 권장: 백엔드 리더보드 API에 `period` 파라미터 지원 추가
 - 심각도: Low
-

C. 번들/코드 분할 — 5건

[BD-H-01] 대시보드 페이지 — 516줄 단일 컴포넌트 (미수정, 4차 연속, 심각도 상향)

- **현상:** dashboard/page.tsx 가 516줄로 실시간 가격, WebSocket 배치, AI 분석 모달(425-513줄, 약 90줄), 스포트라이트 검색, 탭 상태, 관심종목 등 모든 로직을 포함. TODO 주석(RD-M-02)으로 분리 필요성이 명시되어 있으나 4차 연속 미수행. 9차 강화 기준(200줄)에 의해 Critical 후보이나 High로 상향
- **위치:** frontend/src/app/(main)/dashboard/page.tsx:1-516
- **영향:** AI 분석 모달 미사용 시에도 90줄의 JSX+상태+이벤트 핸들러가 초기화됨. 코드 스플리팅 미적용
- **권장:** (1) AI 분석 모달을 dynamic(() => import(...)) 으로 분리 (2) WebSocket 배치 로직을 useLivePrices 커스텀 훅으로 추출 (3) 탭/기간 필터를 별도 훅으로 추출
- **심각도:** High

[BD-M-01] 주문 페이지 — 894줄 단일 파일에 AnalysisTab + OrdersPage 동거

- **현상:** orders/page.tsx 가 894줄로, StatusDropdown (44-108줄), AnalysisTab (111-344줄), OrdersPage (346-894줄) 세 컴포넌트를 단일 파일에 포함. AnalysisTab은 별도 컴포넌트로 추출되었으나 dynamic import 미적용으로 분석 탭 미사용 시에도 코드가 로드됨. 9차 강화 기준(200줄) 대폭 초과
- **위치:** frontend/src/app/(main)/orders/page.tsx:1-894
- **영향:** 주문 목록만 조회 시 분석 탭의 차트 로직(시간 분포, 심볼별 통계 등)이 불필요하게 번들에 포함
- **권장:** AnalysisTab을 별도 파일로 분리 후 dynamic(() => import(...)) 으로 지연 로드. StatusDropdown 도 공유 UI 컴포넌트로 추출
- **심각도:** Medium

[BD-M-02] 리더보드 페이지 — 637줄 단일 컴포넌트

- **현상:** leaderboard/page.tsx 가 637줄로, 기간 필터, 정렬, FLIP 애니메이션, 팔로우/카피트레이드 모달, 사용자 프로필 모달, 언팔로우 확인 모달 등 모든 로직을 포함. 9차 강화 기준(200줄) 3배 초과
- **위치:** frontend/src/app/(main)/leaderboard/page.tsx:1-637
- **영향:** 초기 렌더 시 카피트레이드/프로필 모달 등 미사용 코드까지 전부 로드
- **권장:** 리더보드 테이블 행을 LeaderboardRow 컴포넌트로 분리, 카피트레이드/프로필 모달을 dynamic import로 전환
- **심각도:** Medium

[BD-M-03] 마이페이지 — 807줄 단일 컴포넌트

- **현상:** mypage/page.tsx 가 807줄로, 프로필 조회, 비밀번호 변경 모달(자체 포커스 트랩 구현), 알림 설정 토글, 계정 초기화 2단계 확인, 거래 통계 집계 등을 포함. 비밀번호 변경 모달과 계정 초기화 로직이 페이지 로드 시 항상 번들에 포함
- **위치:** frontend/src/app/(main)/mypage/page.tsx:1-807
- **영향:** 페이지 진입 시 비밀번호 변경/계정 초기화 코드가 불필요하게 로드
- **권장:** 비밀번호 변경 모달, 계정 초기화 UI를 각각 별도 컴포넌트로 분리 후 dynamic import
- **심각도:** Medium

[BD-L-01] PortfolioAnalytics — recharts 직접 import로 번들 포함

- **현상:** PortfolioAnalytics.tsx (904줄)이 recharts의 PieChart, LineChart, BarChart 등을 직접 import. 이 컴포넌트 자체는 portfolio/page.tsx 에서 dynamic(() => import(...)) 으로 코드 스플리팅되어 있으므로 초기 번들에는 미포함. 그러나 904줄 단일 컴포넌트로 9차 기준(200줄) 4.5배 초과
- **위치:** frontend/src/components/portfolio/PortfolioAnalytics.tsx:1-904
- **영향:** analytics 탭 로드 시 904줄 전체가 한 번에 파싱. 각 분석 섹션(자산 배분, 일별 손익, 위험 지표 등)이 항상 로드됨
- **권장:** 각 분석 섹션(PieChart, DailyPnL, RiskMetrics 등)을 별도 컴포넌트로 분리하여 모듈화. dynamic import까지는 현재 구조상 불필요하나 단일 책임 원칙 위반
- **심각도:** Low

D. 캐싱/메모리 관리 — 3건

[CM-M-01] CandlestickChart — chartType 변경 시 8개 시리즈 전체 재생성 (미수정, 3차 연속)

- **현상:** CandlestickChart.tsx:319-404 에서 chartType 변경 시 기존 시리즈 전부 제거 후 재생성. 캔들 ↔ 라인 전환 시 메인 시리즈뿐 아니라 볼륨, SMA 3개, 볼린저 밴드 3개까지 총 8개 시리즈를 destroy/recreate
- **위치:** frontend/src/components/chart/CandlestickChart.tsx:319-404
- **영향:** 차트 타입 전환 시 눈에 띄는 깜빡임과 지연 (시리즈 생성 → 데이터 세팅의 2단계 렌더)
- **권장:** 메인 시리즈만 교체하고 보조 지표 시리즈는 유지, 또는 두 시리즈를 모두 생성 후 visible 토글로 전환
- **심각도:** Medium

[CM-M-02] 리더보드 prevRankMap — 기간/정렬 변경 시 미초기화 (미수정)

- **현상:** leaderboard/page.tsx:235 의 prevRankMap.current 가 컴포넌트 생애 동안 이전 순위 데이터를 계속 유지. period 나 sortMode, investedOnly, copyTradeOnly 변경 시에도 초기화되지 않아 잘못된 순위 변동 애니메이션 발생. 8차에서 이미 지적되었으나 미수정
- **위치:** frontend/src/app/(main)/leaderboard/page.tsx:235
- **영향:** 필터 변경 후 순위 변동 인디케이터가 실제와 다르게 표시. FLIP 애니메이션이 의미 없는 이동을 수행
- **권장:** period, sortMode, investedOnly, copyTradeOnly 변경 시 prevRankMap.current = new Map() 호출
- **심각도:** Medium

[CM-L-01] 리더보드 sortedLeaderboard — 필터 변경 시 전체 재정렬 (미수정)

- **현상:** leaderboard/page.tsx:212-230 의 sortedLeaderboard useMemo가 investedOnly, copyTradeOnly, sortMode, copyTradingUserIds 등 6개 의존성을 가짐. 토글 변경마다 전체 배열을 filter → sort → map으로 재계산
- **위치:** frontend/src/app/(main)/leaderboard/page.tsx:212-230
- **영향:** 사용자 100명+ 시 불필요한 정렬 반복. 단, 현재 limit: 100 으로 100명 미만이므로 실질적 영향은 제한적
- **권장:** 정렬 결과를 캐싱하고 필터만 별도 useMemo로 분리
- **심각도:** Low

E. DB/서버 성능 — 2건

[DB-M-01] 관리자 체결 감사 — 사용자명 enrichment N+1 패턴 (미수정, 3차 연속)

- **현상:** order-proxy.controller.ts:162-179 에서 체결 내역 조회 후 사용자명을 enrichment하기 위해 POST /users/by-ids 호출. 현재는 한번의 배치 호출이지만, 체결 내역의 userId가 다수(100+)일 경우 대량 ID 배열 전송. userMap 결과가 캐싱되지 않아 동일 사용자에게 대해 매 요청마다 반복 조회
- **위치:** backend/services/api-gateway/src/proxy/order-proxy.controller.ts:162-179
- **영향:** 관리자 페이지 체결 내역 조회 시 추가 네트워크 홉, userId 100개+ 시 요청 크기 증가
- **권장:** Redis 캐싱으로 userId → username 매핑 캐시 (TTL 5분), 또는 order-engine DB에 사용자명 비정규화 저장
- **심각도:** Medium

[DB-L-01] 거래 내역(transactions) — limit=200 하드코딩 (미수정)

- **현상:** useTransactions(limit = 200) 혹에서 기본 200건을 가져옴. PortfolioHistoryChart 에서 사용하지만, 기간 필터('1W' 등)가 적용되어도 항상 200건을 전부 가져온 후 클라이언트에서 기간 필터링 수행
 - **위치:** frontend/src/hooks/usePortfolio.ts:240
 - **영향:** 1주 조회 시에도 불필요한 90일+ 거래 데이터 전송
 - **권장:** 기간에 따라 limit 또는 since 파라미터를 동적으로 조정
 - **심각도:** Low
-

F. WebSocket/실시간 처리 — 1건

[WS-M-01] useWebSocket — symbols 배열 변경 시 전체 목록 재전송 (미수정, 3차 연속)

- 현상: useWebSocket.ts:196-198 에서 symbols 변경 시 socket.emit('subscribe', { symbols }) 로 전체 심볼 목록을 재전송. unsubscribe는 diff 기반(removed 계산 후 개별 unsubscribe)이지만, subscribe는 항상 전체 목록을 전송
 - 위치: frontend/src/hooks/useWebSocket.ts:194-198
 - 영향: 심볼 100개+ 시 관심종목 1개 추가/제거 시에도 전체 100개 심볼 재전송
 - 권장: subscribe도 diff 기반으로 신규 추가된 심볼만 전송: `const added = symbols.filter(s => !prevSymbolsRef.current.includes(s))` 후 added만 전송
 - 심각도: Medium
-

G. 리스트/가상화 — 2건

[LV-M-01] 체결 내역 탭 — 전체 filteredTrades를 가상화 없이 렌더링

- 현상: orders/page.tsx:824 에서 filteredTrades.map() 으로 전체 체결 내역을 DOM에 직접 렌더링. useTradeHistory() 가 limit 없이 전체 데이터를 가져오므로, 체결 내역 100건 이상 시 DOM 노드가 비례 증가. 페이지네이션 미적용, 가상화(windowing) 미적용
- 위치: frontend/src/app/(main)/orders/page.tsx:824
- 영향: 체결 내역 500건+ 시 DOM 과다 생성으로 스크롤 성능 저하 및 메모리 증가
- 권장: (1) 클라이언트 측 페이지네이션 추가 (Pagination 컴포넌트 활용) 또는 (2) react-virtual/tanstack-virtual로 리스트 가상화
- 심각도: Medium

[LV-M-02] 리더보드 — sortedLeaderboard 최대 100건 가상화 없이 전체 렌더링

- 현상: leaderboard/page.tsx:469 에서 sortedLeaderboard.map() 으로 최대 100명의 사용자를 가상화 없이 전체 렌더링. 각 행에 팔로우/카피 트레이드 버튼, 프로필 모달 트리거, FLIP 애니메이션 ref 등 복잡한 렌더 로직 포함. 9차 강화 기준(20건 이상 가상화 필수) 해당
 - 위치: frontend/src/app/(main)/leaderboard/page.tsx:469-614
 - 영향: 100명 렌더 시 각 행의 ref 콜백 + 버튼 이벤트 핸들러가 100개씩 생성. FLIP 애니메이션과의 충돌 가능성
 - 권장: FLIP 애니메이션과 호환되는 가상화 라이브러리 적용, 또는 "더 보기" 버튼으로 초기 표시 20명 → 점진적 확장
 - 심각도: Medium
-

H. 기타 — 3건

[ETC-M-01] 뉴스 AI 분석 — 대시보드와 뉴스 페이지에서 AI 분석 모달 코드 중복

- 현상: dashboard/page.tsx:425-513 과 news/page.tsx:370-498 에서 거의 동일한 AI 분석 모달 UI(로딩, 에러, 결과 표시, 섹션 분석)를 각각 구현. 뉴스 페이지는 marketOutlook , riskFactors 추가 섹션만 차이. handleAiAnalysis 콜백도 동일한 200건 조회 → 필터 → AI 요청 패턴을 중복 구현
- 위치: frontend/src/app/(main)/dashboard/page.tsx:425-513 , frontend/src/app/(main)/news/page.tsx:370-498
- 영향: 동일 버그를 두 곳에서 각각 수정해야 하며, 번들 크기 낭비
- 권장: <AiNewsAnalysisModal /> 공유 컴포넌트로 추출하고, handleAiAnalysis 로직을 useAiAnalysis() 커스텀 훅으로 추출
- 심각도: Medium

[ETC-M-02] useTradeHistory 호출 중복 — 마이페이지/주문 페이지 동시 활성화 시 중복 폴링

- **현상:** `useTradeHistory()` 가 `orders/page.tsx:369` 과 `mypage/page.tsx:139` 에서 각각 호출됨. 동일한 `queryKey` `['trades', 'history']` 를 사용하므로 TanStack Query의 `dedup` 메커니즘으로 단일 요청으로 병합됨. 그러나 두 페이지가 동시 활성화되지 않더라도 각각 10초 폴링(`refetchInterval: 10000`)이 독립 실행. 마이페이지에서는 통계 집계만 필요하므로 10초 폴링 불필요
- **위치:** `frontend/src/app/(main)/mypage/page.tsx:139` , `frontend/src/hooks/useOrders.ts:214`
- **영향:** 마이페이지 체류 시 불필요한 10초 폴링으로 서버 부하
- **권장:** 마이페이지에서는 `refetchInterval` 을 비활성화하거나, `staleTime`을 길게 설정하여 캐시된 데이터 활용
- **심각도:** Medium

[ETC-L-01] AssetList FLIP 애니메이션 — `void el.offsetHeight` 강제 리플로우

- **현상:** `AssetList.tsx:200` 에서 `void el.offsetHeight` 로 강제 리플로우를 발생시켜 FLIP 애니메이션을 구현. 리더보드(`leaderboard/page.tsx:274`)에서는 `void movedEls[0].offsetHeight` 로 단일 리플로우만 발생시키는 최적화가 적용되었으나, `AssetList`에서는 `paged.forEach` 루프 내에서 개별 요소마다 리플로우 발생 가능
- **위치:** `frontend/src/components/market/AssetList.tsx:199-200`
- **영향:** 순위 변동이 많을 때 다수의 강제 리플로우로 프레임 드롭 발생 가능. 단, 3초 쓰로틀로 빈도는 제한됨
- **권장:** 리더보드와 동일하게 `moved` 엘리먼트를 수집한 후 단일 리플로우 → `requestAnimationFrame`으로 배치 처리
- **심각도:** Low

종합 평가

9차 성능 감사에서 총 25건의 이슈를 발견하였습니다. 8차 대비 주문 분석 탭의 컴포넌트 분리(BD-L-01)가 수정되었으나, PF-H-01(뉴스 200건 과잉 조회), API-H-01(뉴스 필터 200건 일괄 조회), BD-M-01(대시보드 516줄 단일 컴포넌트) 등 핵심 이슈 3건이 3~4차 연속 미수정 상태입니다.

9차 강화 기준(200줄 초과 플러그)에 의해 새로운 번들/코드 분할 이슈가 추가 발견되었습니다:

- 주문 페이지 894줄 (BD-M-01)
- 리더보드 637줄 (BD-M-02)
- 마이페이지 807줄 (BD-M-03)
- PortfolioAnalytics 904줄 (BD-L-01)

가장 시급한 개선 항목은:

1. **AI 분석 뉴스 200건 과잉 조회** (PF-H-01) — 3차 연속 미수정. 서버 측 날짜 필터 적용으로 네트워크/서버 부하 제거
2. **뉴스 필터 200건 일괄 조회** (API-H-01) — 3차 연속 미수정. 서버 측 검색/날짜 필터 파라미터 추가
3. **대시보드 516줄 단일 컴포넌트** (BD-H-01) — 4차 연속 미수정. AI 모듈 분리 및 WebSocket 배치 로직 혹 추출

전체적으로 "전체 조회 + 클라이언트 필터" 패턴이 뉴스/체결 내역 영역에 집중적으로 남아있으며, 200줄 초과 컴포넌트가 다수 존재하여 코드 유지보수성과 번들 효율 개선이 필요합니다. AI 분석 모듈의 대시보드/뉴스 중복 코드를 공유 컴포넌트로 통합하면 약 200줄의 중복을 제거할 수 있습니다.